UNIVERSIDAD DE LOS ANDES

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Laboratorio #3: Análisis Capa de Transporte y Sockets

ISIS 2311 – REDES Y SERVICIOS DE COMUNICACIONES

Profesor: Yezid Donoso
Asist. de laboratorio: Nathalia Quiroga

Grupo 5

Miguel Cabrera Escobar - 202020756
Alejandro Cruz Acevedo - 201912149
Sofía Losada Muñóz - 202221008

2026-10

Índice

1. Introducción **2**

2. Marco Teórico **2**
- 2.1. Sockets . . . 2
- 2.2. TCP (Transmission Control Protocol) . . . 2
- 2.3. UDP (User Datagram Protocol) . . . 2
- 2.4. Modelo Publicación–Suscripción . . . 2

3. Configuración del Entorno **3**
- 3.1. Estructura de Archivos . . . 3
- 3.2. Compilación . . . 3

4. Implementación **3**
- 4.1. Versión TCP . . . 3
  - 4.1.1. Broker TCP . . . 3
  - 4.1.2. Publisher TCP . . . 4
  - 4.1.3. Subscriber TCP . . . 4
- 4.2. Versión UDP . . . 5
  - 4.2.1. Broker UDP . . . 5
  - 4.2.2. Publisher UDP . . . 5
  - 4.2.3. Subscriber UDP . . . 5

5. Pruebas y Resultados **5**
- 5.1. Escenario de Prueba . . . 5
- 5.2. Resultados TCP . . . 5
- 5.3. Resultados UDP . . . 6
- 5.4. Análisis con Wireshark . . . 7
  - 5.4.1. Captura TCP (`tcp_pubsub.pcap`) . . . 7
  - 5.4.2. Captura UDP (`udp_pubsub.pcap`) . . . 9

6. Comparación TCP vs. UDP **12**

7. Análisis y Preguntas **13**
- 7.1. Escalabilidad del broker con 100 publicadores . . . 13
- 7.2. Pérdida de mensajes críticos en UDP . . . 13
- 7.3. Protocolo más adecuado para seguimiento en vivo . . . 13
- 7.4. Overhead observado en Wireshark . . . 13
- 7.5. Mensajes desordenados en UDP . . . 13
- 7.6. Escalabilidad con múltiples suscriptores . . . 13
- 7.7. Caída inesperada del broker . . . 14
- 7.8. Sincronización de actualizaciones críticas . . . 14
- 7.9. Uso de CPU y memoria en el broker . . . 14
- 7.10. Diseño de un sistema real para millones de usuarios . . . 14

8. Conclusiones **15**

Referencias **16**

Archivos de Captura (.pcap) **16**

1. Introducción

En este laboratorio se implementa y analiza un sistema de comunicación basado en el modelo publicación–suscripción utilizando sockets en x86-64 Assembly (NASM) sobre Linux. El sistema simula una plataforma de noticias deportivas en tiempo real, en la que publicadores envían actualizaciones de partidos a un broker central, quien las redistribuye a los suscriptores interesados. El objetivo principal es comparar el comportamiento de los protocolos de la capa de transporte TCP y UDP en términos de confiabilidad, orden de entrega, pérdida de mensajes y overhead de protocolo, complementando el análisis con capturas de tráfico obtenidas mediante Wireshark. Como valor adicional, la implementación completa se realizó en lenguaje ensamblador x86-64, interactuando directamente con el kernel de Linux mediante instrucciones `syscall`, lo que permite observar con mayor detalle el funcionamiento de los sockets a nivel de sistema operativo.

2. Marco Teórico

2.1. Sockets

Un socket es una abstracción del sistema operativo que representa un extremo de comunicación entre dos procesos, ya sea en la misma máquina o a través de una red. Se identifica de forma única mediante un par `(dirección IP, puerto)`, por ejemplo `192.168.1.10:5000`, donde la IP localiza la máquina y el puerto distingue el proceso o servicio específico dentro de ella. En C, los sockets se crean con la llamada al sistema `socket()`, se configuran con `bind()` y `connect()` o `listen()/accept()`, y se usan para enviar y recibir datos con `send()/recv()` (TCP) o `sendto()/recvfrom()` (UDP).

2.2. TCP (Transmission Control Protocol)

TCP es un protocolo orientado a la conexión que opera en la capa de transporte. Antes de intercambiar datos, los dos extremos establecen un canal confiable mediante el *three-way handshake*: el cliente envía un segmento `SYN`, el servidor responde con `SYN-ACK` y el cliente confirma con `ACK`. Una vez establecida la conexión, TCP garantiza entrega confiable mediante retransmisiones, preserva el orden de los segmentos con números de secuencia, y regula la velocidad de transmisión a través de mecanismos de control de flujo (*ventana deslizante*) y control de congestión. Estas garantías implican un mayor overhead en cabeceras (mínimo 20 bytes) y latencia adicional frente a UDP.

2.3. UDP (User Datagram Protocol)

UDP es un protocolo no orientado a la conexión. Cada mensaje se transmite como un datagrama independiente sin establecer ni mantener ningún canal previo, lo que elimina el overhead del handshake y reduce la cabecera a tan solo 8 bytes. Sin embargo, UDP no ofrece garantías: los datagramas pueden perderse, duplicarse o llegar en un orden distinto al que fueron enviados, y no existe ningún mecanismo de retransmisión ni confirmación a nivel de protocolo. Esta simplicidad lo hace ideal para aplicaciones donde la velocidad y el bajo overhead son prioritarios sobre la confiabilidad, como streaming de video, videojuegos en línea o transmisiones de audio en tiempo real.

2.4. Modelo Publicación–Suscripción

El modelo publicación–suscripción (*pub/sub*) es un patrón de comunicación asíncrona en el que los productores de información (*publishers*) no envían mensajes directamente a los consumidores, sino a un intermediario central denominado *broker*. Los consumidores (*subscribers*) se registran en el broker indicando los temas o canales de su interés, y el broker se encarga de distribuir cada mensaje recibido únicamente a los suscriptores correspondientes. Este desacoplamiento entre publicadores y suscriptores permite que el sistema escale con facilidad: agregar nuevos publicadores o suscriptores no requiere modificar los demás componentes, y el broker puede gestionar múltiples flujos de información de forma centralizada.

3. Configuración del Entorno

3.1. Estructura de Archivos

El laboratorio se desarrolló en un entorno Ubuntu 24.04 corriendo sobre WSL2 (Windows Subsystem for Linux) en Windows 11. Se creó el directorio `~/lab3` con seis archivos fuente en lenguaje ensamblador x86-64 (NASM), organizados en dos grupos según el protocolo de transporte utilizado:

- `broker_tcp.asm` — Broker central para la versión TCP
- `publisher_tcp.asm` — Publicador para la versión TCP
- `subscriber_tcp.asm` — Suscriptor para la versión TCP
- `broker_udp.asm` — Broker central para la versión UDP
- `publisher_udp.asm` — Publicador para la versión UDP
- `subscriber_udp.asm` — Suscriptor para la versión UDP

3.2. Compilación

Cada archivo `.asm` se ensambló con NASM generando un archivo objeto `.o` en formato ELF64, y luego se linkeditó con `ld` para producir el ejecutable final. No se utilizó ninguna librería externa — todos los binarios interactúan directamente con el kernel mediante `syscall`.

Los comandos de compilación utilizados fueron:

```
nasm -f elf64 broker_tcp.asm     -o broker_tcp.o     && ld broker_tcp.o     -o
    broker_tcp
nasm -f elf64 publisher_tcp.asm  -o publisher_tcp.o  && ld publisher_tcp.o  -o
    publisher_tcp
nasm -f elf64 subscriber_tcp.asm -o subscriber_tcp.o && ld subscriber_tcp.o -o
    subscriber_tcp
nasm -f elf64 broker_udp.asm     -o broker_udp.o     && ld broker_udp.o     -o
    broker_udp
nasm -f elf64 publisher_udp.asm  -o publisher_udp.o  && ld publisher_udp.o  -o
    publisher_udp
nasm -f elf64 subscriber_udp.asm -o subscriber_udp.o && ld subscriber_udp.o -o
    subscriber_udp
```

4. Implementación

Toda la implementación se realizó en lenguaje ensamblador x86-64 (NASM) sobre Linux. Las llamadas al sistema operativo se realizan directamente mediante la instrucción `syscall`, cargando el número de syscall en `rax` y los argumentos en los registros `rdi`, `rsi`, `rdx`, `r10`, `r8` y `r9` respectivamente. No se utilizaron librerías externas de ningún tipo.

4.1. Versión TCP

4.1.1. Broker TCP

El broker TCP actúa como servidor central. Crea un socket de tipo `SOCK_STREAM` (syscall 41), lo enlaza al puerto 5000 con `bind()` (syscall 49) y queda en escucha con `listen()` (syscall 50). En su bucle principal acepta conexiones con `accept()` (syscall 43) y lee el primer byte enviado por cada cliente para determinar si es un publisher (`'P'`) o un subscriber (`'S'`). Los subscribers se registran en una lista interna; cuando llega un mensaje de un publisher, el broker lo reenvía a todos los subscribers registrados mediante `write()` (syscall 1). El fragmento central del bucle de reenvío es el siguiente:

```
; Leer tipo de cliente (P o S)
mov rax, 0          ; syscall read
mov edi, [client_fd]
lea rsi, [type_buf]
mov rdx, 1
syscall

cmp byte [type_buf], 0x50   ; 'P' = Publisher
je  handle_publisher
cmp byte [type_buf], 0x53   ; 'S' = Subscriber
je  handle_subscriber
```

4.1.2. Publisher TCP

El publisher TCP crea un socket `SOCK_STREAM` y se conecta al broker mediante `connect()` (syscall 42). Antes de enviar mensajes, transmite el byte `'P'` para identificarse. Luego envía 10 mensajes de noticias deportivas uno por uno con una pausa de 1 segundo entre cada uno (implementada con `nanosleep`, syscall 35):

```
; Identificarse como Publisher
mov rax, 1          ; syscall write
mov edi, [sock_fd]
lea rsi, [type_pub] ; byte 'P'
mov rdx, 1
syscall

; Enviar mensajes con delay de 1 segundo
sub rsp, 16
mov qword [rsp],   1    ; segundos
mov qword [rsp+8], 0    ; nanosegundos
mov rax, 35             ; syscall nanosleep
mov rdi, rsp
mov rsi, 0
syscall
add rsp, 16
```

4.1.3. Subscriber TCP

El subscriber TCP se conecta al broker con `connect()` y se identifica enviando el byte `'S'`. Luego entra en un bucle infinito de `read()` (syscall 0) esperando mensajes del broker. Cada mensaje recibido se imprime por stdout con `write()` (syscall 1). La conexión persiste hasta que el broker la cierra:

```
recv_loop:
    mov rax, 0          ; syscall read
    mov edi, [sock_fd]
    lea rsi, [buf]
    mov rdx, 1024
    syscall

    cmp rax, 0          ; 0 bytes = conexion cerrada
    jle recv_done

    mov r12, rax        ; guardar longitud
    ; imprimir mensaje recibido...
    jmp recv_loop
```

4.2. Versión UDP

4.2.1. Broker UDP

El broker UDP crea un socket `SOCK_DGRAM` (syscall 41 con `rsi=2`) y lo enlaza al puerto 5001. A diferencia de TCP, no existe `listen()` ni `accept()` — el broker simplemente bloquea en `recvfrom()` (syscall 45) esperando datagramas de cualquier origen. Cuando recibe un datagrama con el byte `'S'`, registra la dirección IP y puerto del remitente como subscriber. Cuando recibe un datagrama con `'P'`, lo reenvía a todos los subscribers con `sendto()` (syscall 44). Para simular las características reales de UDP, el broker descarta intencionalmente 1 de cada 4 mensajes mediante un contador:

```
handle_message:
    inc dword [drop_counter]
    mov eax, [drop_counter]
    cmp eax, 4
    jne forward_message     ; si no es multiplo de 4, reenviar

    mov dword [drop_counter], 0
    ; imprimir mensaje de descarte...
    jmp recv_loop           ; descartar sin reenviar
```

4.2.2. Publisher UDP

El publisher UDP crea un socket `SOCK_DGRAM` y envía cada mensaje directamente al broker con `sendto()` (syscall 44) sin establecer ninguna conexión previa. Cada mensaje incluye el prefijo `'P'` para que el broker lo identifique como mensaje de publicación:

```
; sendto() sin conexion previa -- UDP puro
mov rax, 44             ; syscall sendto
mov edi, [sock_fd]
; rsi = puntero al mensaje, rdx = longitud
mov r10, 0
lea r8,  [broker_addr]  ; destino
mov r9,  16             ; tamano sockaddr_in
syscall
```

4.2.3. Subscriber UDP

El subscriber UDP se enlaza a su propio puerto (5002) con `bind()` para poder recibir datagramas entrantes, luego envía un datagrama de registro con el byte `'S'` al broker. A partir de ese momento queda en un bucle de `recvfrom()` esperando mensajes. Al no existir conexión persistente, el subscriber no sabe cuándo termina la transmisión y permanece en escucha indefinidamente hasta recibir `Ctrl+C`.

5. Pruebas y Resultados

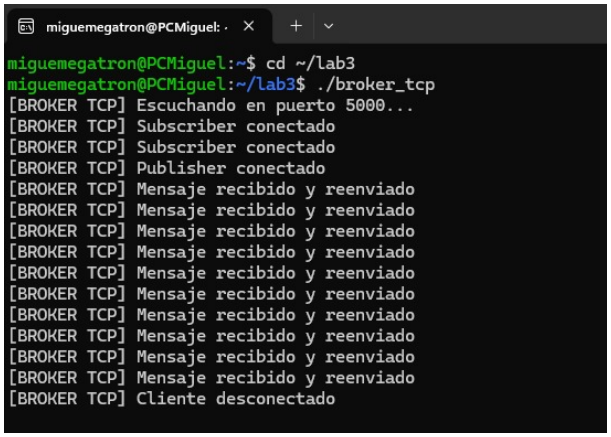
5.1. Escenario de Prueba

Las pruebas se ejecutaron en una única máquina (localhost, `127.0.0.1`) con WSL2 sobre Windows 11. Se abrieron cuatro terminales Ubuntu simultáneas. El escenario consistió en un broker, dos suscriptores y un publicador corriendo al mismo tiempo. Los puertos utilizados fueron el 5000 para TCP y el 5001 para UDP (el subscriber UDP adicionalmente escucha en el puerto 5002). El publisher envió exactamente 10 mensajes con un intervalo de 1 segundo entre cada uno.

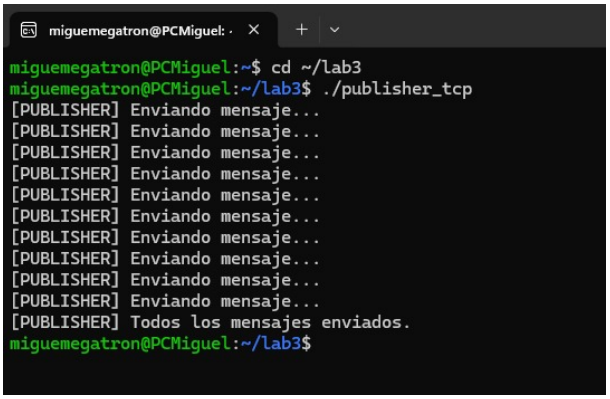
5.2. Resultados TCP

En la versión TCP, los 10 mensajes enviados por el publisher llegaron completos y en orden a ambos subscribers. El broker registró la conexión de los dos subscribers antes de recibir al publisher, y procesó y

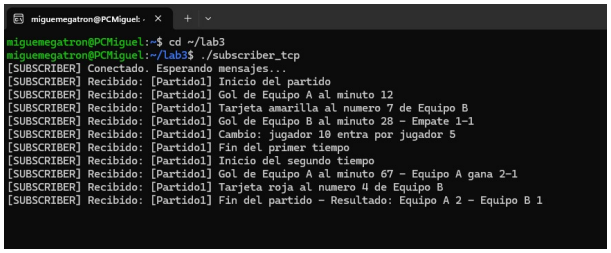
reenvió los 10 mensajes sin ninguna pérdida. Al terminar la transmisión, el broker detectó la desconexión del publisher y lo reportó en consola.



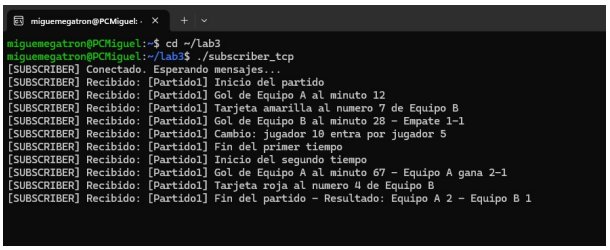
(a) Broker TCP: 10 mensajes recibidos y reenviados



(b) Publisher TCP: 10 mensajes enviados



(c) Subscriber 1 TCP: 10 mensajes recibidos en orden



(d) Subscriber 2 TCP: 10 mensajes recibidos en orden

Figura 1: Resultados de la ejecución TCP — broker, publisher y dos subscribers

5.3. Resultados UDP

En la versión UDP, el broker recibió los 10 datagramas del publisher pero descartó 2 de ellos (los mensajes 4 y 8, correspondientes a cada cuarto datagrama), simulando la pérdida característica de UDP. Los subscribers recibieron únicamente los 8 mensajes restantes, sin posibilidad de recuperar los perdidos ya que UDP no implementa retransmisiones. Adicionalmente, dado que UDP no mantiene estado de conexión, el subscriber permaneció en escucha indefinidamente tras el fin de la transmisión.

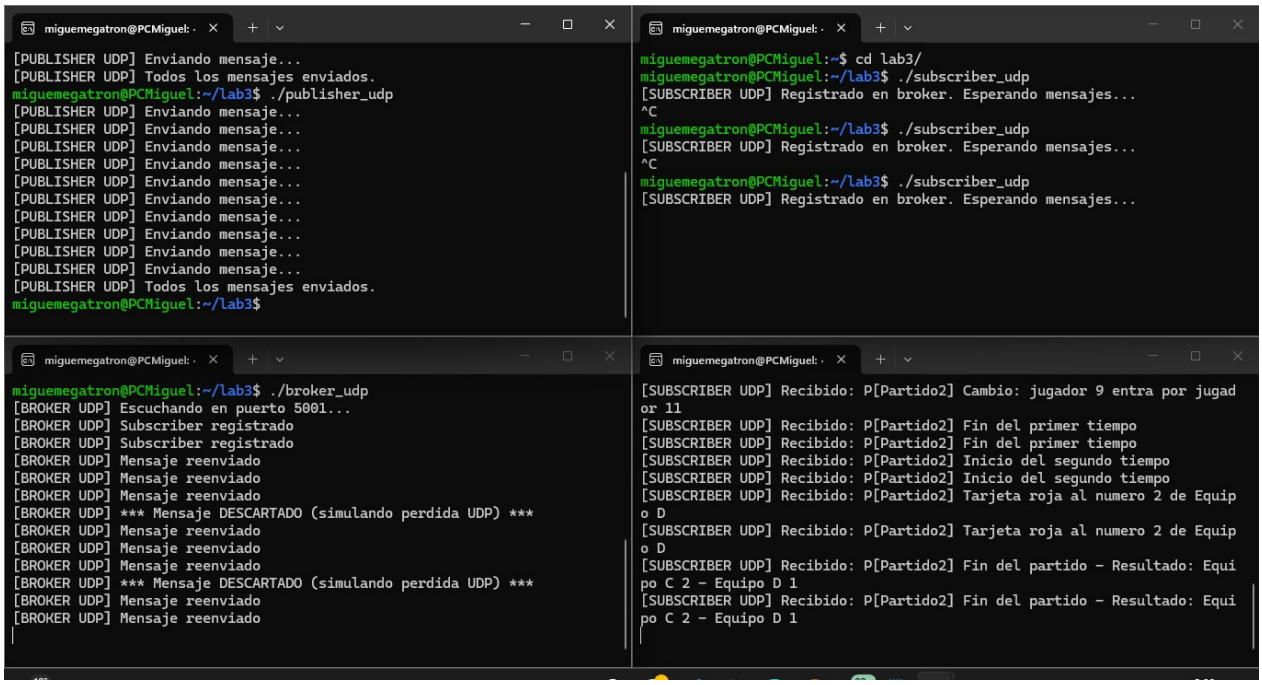


Figura 2: Resultados UDP: broker con mensajes descartados y subscriber con mensajes faltantes

5.4. Análisis con Wireshark

Las capturas de tráfico se realizaron utilizando `tcpdump` en la interfaz de loopback (`lo`), capturando el tráfico en el puerto 8080. La captura se inició antes de activar cualquier componente del sistema para registrar el ciclo de vida completo de la comunicación. Los archivos `.pcap` generados fueron analizados posteriormente en Wireshark.

5.4.1. Captura TCP (`tcp_pubsub.pcap`)

La captura TCP fue obtenida ejecutando el siguiente comando antes de iniciar cualquier componente:

```
sudo tcpdump -i lo port 8080 -w captura_final.pcap
```

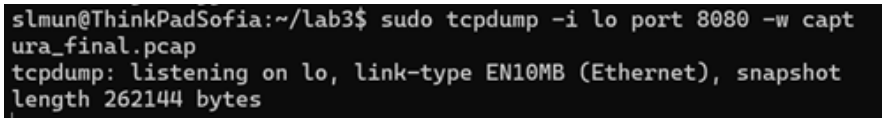


Figura 3: Inicio de captura de tráfico TCP con `tcpdump` sobre la interfaz loopback

A continuación se activaron el broker, el subscriber y el publisher desde terminales WSL independientes:

```
slmun@ThinkPadSofia:~/lab3$ gcc broker_tcp.c -o broker_tcp -lpth
read
slmun@ThinkPadSofia:~/lab3$ ./broker_tcp
Broker TCP deportivo iniciado en puerto 8080...
```

(a) Activación del Broker TCP

```
slmun@ThinkPadSofia:~/lab3$ gcc subscriber_tcp.c -o sub_tcp
slmun@ThinkPadSofia:~/lab3$ ./sub_tcp
Suscrito con éxito al partido: Colombia vs Brasil
Esperando actualizaciones en vivo...
```

(b) Activación del Subscriber TCP

```
slmun@ThinkPadSofia:~/lab3$ gcc publisher_tcp.c -o pub_tcp
slmun@ThinkPadSofia:~/lab3$ ./pub_tcp
Conectado al Broker. Enviando noticias...
Enviado: Evento deportivo #1: ¡Acción en el campo!
Enviado: Evento deportivo #2: ¡Acción en el campo!
Enviado: Evento deportivo #3: ¡Acción en el campo!
Enviado: Evento deportivo #4: ¡Acción en el campo!
Enviado: Evento deportivo #5: ¡Acción en el campo!
Enviado: Evento deportivo #6: ¡Acción en el campo!
Enviado: Evento deportivo #7: ¡Acción en el campo!
Enviado: Evento deportivo #8: ¡Acción en el campo!
Enviado: Evento deportivo #9: ¡Acción en el campo!
Enviado: Evento deportivo #10: ¡Acción en el campo!
Transmisión finalizada.
slmun@ThinkPadSofia:~/lab3$
```

(c) Activación del Publisher TCP: 10 mensajes enviados

Figura 4: Activación secuencial de los componentes TCP durante la captura

Flujo general de la captura. La captura de tráfico evidencia un flujo de comunicación íntegro basado en TCP a través de la interfaz de loopback (`127.0.0.1`). En la columna de información de Wireshark se observa una secuencia lógica y ordenada de eventos que abarca desde el establecimiento de la sesión hasta el cierre final de la conexión.

Cierre de conexión y Three-Way Handshake. En la Figura 5 se observa la captura completa. Los renglones iniciales muestran los paquetes `[FIN, ACK]` que indican el cierre ordenado de la conexión. En los renglones 11, 12 y 13 se identifica el three-way handshake: el paquete 11 presenta el bit `SYN` activado indicando la intención de conexión, el paquete 12 responde con `[SYN, ACK]`, y el paquete 13 completa el handshake con `[ACK]`. El panel inferior muestra el detalle del paquete seleccionado con el flag `FIN` activado (*Finish: Set*), confirmando el inicio del procedimiento de cierre. Este intercambio garantiza la liberación ordenada de recursos y evita que los puertos queden en estado de espera.

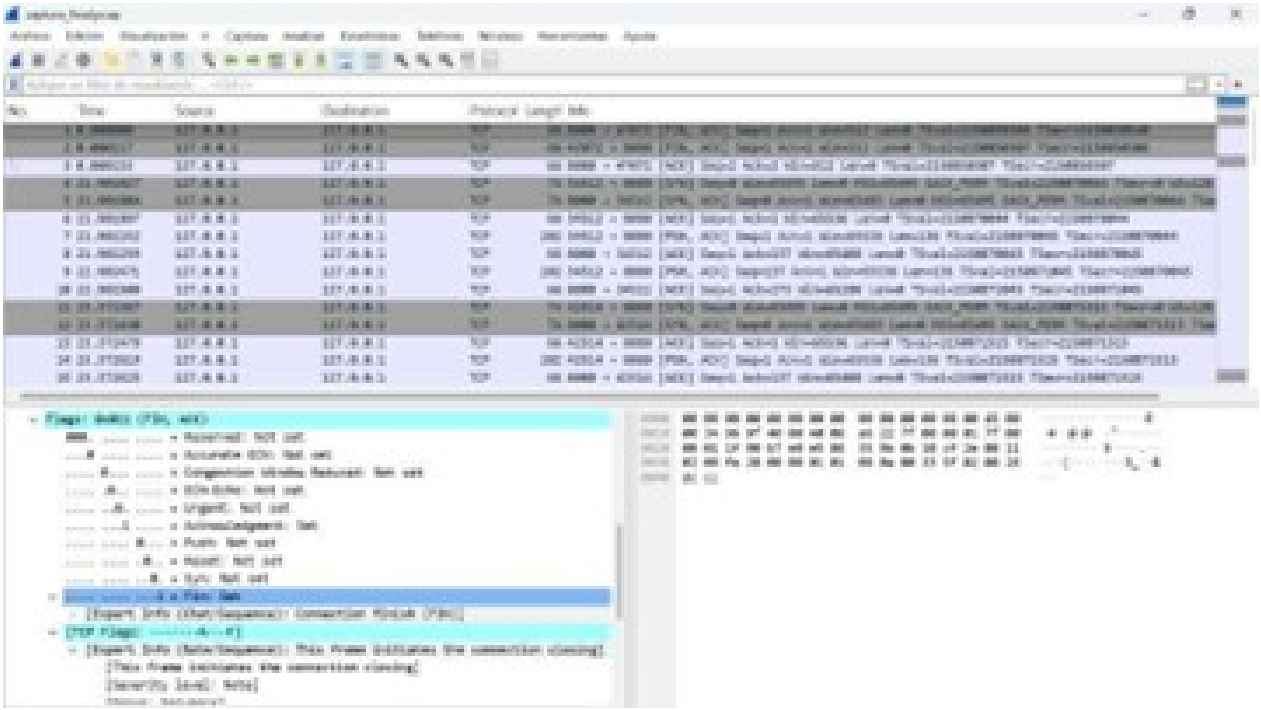


Figura 5: Captura Wireshark TCP: cierre de conexión con flag FIN y ciclo de vida completo

Durante el three-way handshake también se negocian parámetros críticos como el *Maximum Segment Size* (MSS) y el *Window Scale*, visibles en el panel de detalle. Esto garantiza que ambos extremos estén sincronizados y listos para una transferencia de datos confiable antes de intercambiar cualquier payload.

Overhead del protocolo TCP. El análisis de la jerarquía de protocolos en Wireshark (Figura 6) revela que TCP representa el **27.6 %** del total de bytes transmitidos: 1632 bytes distribuidos en 50 paquetes. Este overhead refleja el costo operativo de garantizar confiabilidad — números de secuencia, ACKs, gestión de ventanas y el handshake inicial. En contraste, la carga útil real (los mensajes deportivos) representa una fracción menor del tráfico total.

Wireshark · Estadísticas de jerarquía de protocolo · captura_final.pcap

Protocolo	Porcentaje de paquetes	Paquetes	Porcentaje de bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	50	100.0	5916	1483	0	0	0	50
Ethernet	100.0	50	11.8	700	175	0	0	0	50
Internet Protocol Version 4	100.0	50	16.9	1000	250	0	0	0	50
Transmission Control Protocol	100.0	50	27.6	1632	409	50	1632	409	50

Figura 6: Estadísticas de jerarquía de protocolos TCP: TCP representa el 27.6 % del tráfico (1632 bytes, 50 paquetes)

5.4.2. Captura UDP (`udp_pubsub.pcap`)

Para la captura UDP se utilizó el mismo procedimiento con `tcpdump`. Se activaron los componentes UDP en el siguiente orden: broker, subscriber y publisher.

```
slmun@ThinkPadSofia:~/lab3$ ./broker_udp
Broker UDP deportivo iniciado en puerto 8080...
```

(a) Activación del Broker UDP

```
slmun@ThinkPadSofia:~/lab3$ ./publisher_udp
Enviado por UDP: UDP Gol #1 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #2 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #3 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #4 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #5 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #6 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #7 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #8 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #9 - ¡Velocidad pura!
Enviado por UDP: UDP Gol #10 - ¡Velocidad pura!
slmun@ThinkPadSofia:~/lab3$
```

(b) Publisher UDP: 10 datagramas enviados sin esperar ACK

```
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Noticia recibida de [Colombia vs Brasil]
Broker UDP: Nueva suscripción para [Colombia vs Brasil]
```

(c) Broker UDP: 10 noticias recibidas y reenviadas

Figura 7: Activación de los componentes UDP durante la captura

Flujo de paquetes UDP. A diferencia de TCP, la captura UDP muestra un flujo significativamente más ligero y directo. Como se observa en la Figura 8, la comunicación inicia inmediatamente con el envío de datagramas — no existe ningún paquete previo de establecimiento de conexión. La ausencia total de `SYN`, `ACK` y `FIN` confirma la naturaleza no orientada a conexión del protocolo. Los 11 paquetes capturados corresponden directamente a los mensajes del sistema, sin ningún overhead de control adicional.

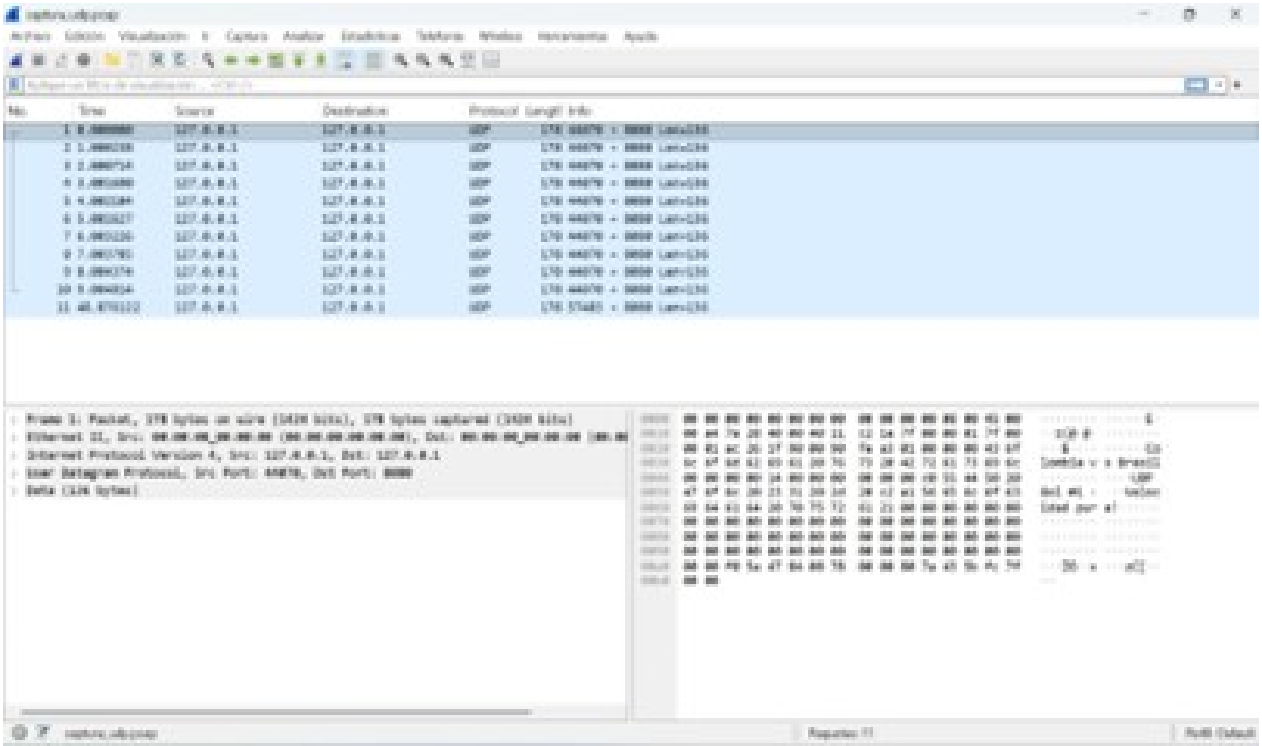


Figura 8: Flujo de paquetes UDP en Wireshark: datagramas independientes sin handshake ni ACKs. Detalle del datagrama con cabecera de 8 bytes y payload de 136 bytes.

Estructura del datagrama UDP. El análisis del paquete No. 1 en el panel inferior de Wireshark permite validar la estructura mínima del protocolo. Se identifican los puertos de origen (44070) y destino (8080), una cabecera de longitud fija de **8 bytes**, y un payload de 136 bytes que contiene íntegro el mensaje deportivo, resultando en un datagrama total de 144 bytes. Esta estructura carece completamente de campos de control como números de secuencia o flags SYN/ACK. El único mecanismo de integridad es el campo *Checksum* (`0xfea3`), que verifica la ausencia de errores de bits pero no ofrece retransmisión en caso de pérdida.

Eficiencia del protocolo UDP. El análisis de jerarquía de protocolos (Figura 9) revela una distribución radicalmente diferente a TCP. Los datos útiles representan el **76.4 %** del total de bytes transmitidos (1496 de 1958 bytes), mientras que UDP solo consume el **4.5 %** (88 bytes) del tráfico total. Con tan solo 11 paquetes para completar la transmisión de 10 mensajes (versus 50 paquetes en TCP), UDP demuestra una eficiencia superior en términos de overhead, a costa de no ofrecer garantías de entrega ni ordenamiento.

Wireshark · Estadísticas de jerarquía de protocolo · captura_udp.pcap

Protocolo	Porcentaje de paquetes	Paquetes	Porcentaje de bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	11	100.0	1958	320	0	0	0	11
Ethernet	100.0	11	7.9	154	25	0	0	0	11
Internet Protocol Version 4	100.0	11	11.2	220	36	0	0	0	11
User Datagram Protocol	100.0	11	4.5	88	14	0	0	0	11
Data	100.0	11	76.4	1496	244	11	1496	244	11

Figura 9: Estadísticas de jerarquía UDP: datos útiles representan el 76.4 % del tráfico vs 4.5 % de overhead UDP

La sincronía entre las terminales y la captura de Wireshark confirma que la arquitectura UDP es altamente eficiente para servicios de noticias en tiempo real. La ausencia de paquetes de control permitió que

prácticamente la totalidad del tráfico capturado correspondiera a la carga útil de los eventos deportivos, en contraste con el overhead significativo observado en TCP.

6. Comparación TCP vs. UDP

Cuadro 1: Comparación de TCP y UDP en el sistema pub/sub

Criterio	**TCP**	**UDP**
Confiabilidad	Garantizada. Todos los mensajes llegaron completos a ambos subscribers. TCP retransmite automáticamente en caso de pérdida.	No garantizada. El broker descartó 2 de 10 mensajes y los subscribers no tuvieron forma de recuperarlos.
Orden de entrega	Garantizado. Los 10 mensajes llegaron en el mismo orden en que fueron enviados, gracias a los números de secuencia TCP.	No garantizado. UDP no numera los datagramas; en redes con congestión podrían llegar desordenados.
Pérdida de mensajes	Ninguna. TCP detecta pérdidas mediante ACKs y retransmite los segmentos no confirmados.	Presente. Se perdieron 2 de 10 mensajes (20 %) sin ningún mecanismo de recuperación a nivel de protocolo.
Overhead de cabeceras	Mayor. Cabecera TCP mínima de 20 bytes por segmento, más el overhead del handshake inicial (3 paquetes) y el cierre de conexión.	Menor. Cabecera UDP fija de solo 8 bytes por datagrama, sin establecimiento ni cierre de conexión.

7. Análisis y Preguntas

7.1. Escalabilidad del broker con 100 publicadores

Con 100 partidos transmitidos simultáneamente, el impacto sería radicalmente distinto según el protocolo. Bajo TCP, el broker mantendría 100 conexiones persistentes activas de forma simultánea, cada una con su propio estado de sesión, buffers de envío y recepción, y control de flujo independiente. Esto implicaría un consumo creciente de memoria y descriptores de archivo. Bajo UDP, el broker simplemente procesaría datagramas entrantes sin mantener estado por cada publicador, lo que reduce significativamente el overhead de memoria. Sin embargo, en UDP el broker podría recibir datagramas de múltiples publicadores en ráfagas simultáneas y, al no tener mecanismo de control de flujo, podría comenzar a descartar datagramas por saturación del buffer del socket del sistema operativo antes de que el propio código los procese.

7.2. Pérdida de mensajes críticos en UDP

Si un gol es descartado por la red antes de llegar al subscriber, ese evento simplemente no existirá para el usuario — no hay ningún mecanismo en UDP que detecte la pérdida ni solicite retransmisión. El subscriber pasaría de "Empate 1-1" directamente a "Equipo A gana 2-1" sin haber visto el gol, generando una experiencia inconsistente e incompleta. TCP maneja mejor este escenario porque cada segmento es confirmado con un ACK; si el ACK no llega dentro de un tiempo límite, TCP retransmite automáticamente el segmento hasta garantizar su entrega, preservando la integridad del flujo de información.

7.3. Protocolo más adecuado para seguimiento en vivo

Basándose en los resultados obtenidos, TCP resulta más adecuado para un sistema de noticias deportivas en tiempo real donde la integridad de la información es crítica. La pérdida del 20 % de mensajes observada en UDP implicaría que los usuarios perderían eventos importantes del partido. Sin embargo, si el sistema evolucionara hacia transmisión de video o audio en tiempo real donde la latencia importa más que la exactitud, UDP con un protocolo de recuperación propio a nivel de aplicación sería preferible.

7.4. Overhead observado en Wireshark

En las capturas de Wireshark se observa que cada segmento TCP incluye una cabecera de al menos 20 bytes con campos de número de secuencia, número de ACK, flags, tamaño de ventana y checksum, además del overhead inicial del three-way handshake (3 paquetes adicionales antes de cualquier dato). En contraste, cada datagrama UDP contiene únicamente 8 bytes de cabecera con los campos de puerto origen, puerto destino, longitud y checksum. Para mensajes pequeños como los del sistema pub/sub, el overhead relativo de TCP es considerablemente mayor, aunque en términos absolutos la diferencia es de solo 12 bytes por mensaje.

7.5. Mensajes desordenados en UDP

Si el subscriber recibiera primero el marcador "2-1" y luego el "1-1", el usuario vería un resultado imposible: el equipo ya ganando antes de haber empatado. Esto rompería completamente la narrativa del partido y generaría confusión. A nivel de aplicación, la solución estándar es agregar un número de secuencia en el propio payload del mensaje (independiente del protocolo de transporte), y en el subscriber mantener un buffer de reordenamiento que espere recibir los mensajes en orden antes de presentarlos al usuario.

7.6. Escalabilidad con múltiples suscriptores

Al aumentar el número de subscribers, ambos protocolos se ven afectados pero de formas distintas. En TCP, cada nuevo subscriber agrega una conexión persistente que el broker debe mantener activa, consumiendo un descriptor de archivo y buffers de kernel por cada una. En UDP, agregar subscribers simplemente agrega una entrada más a la lista de direcciones destino del broadcast, sin overhead de estado adicional. En las pruebas realizadas con 2 subscribers, ambos protocolos funcionaron sin degradación perceptible, pero a escala (decenas o cientos de subscribers) UDP escalaría más eficientemente en términos de recursos del broker.

7.7. Caída inesperada del broker

Bajo TCP, si el broker cae inesperadamente, los subscribers y publishers detectan el cierre de la conexión casi de inmediato mediante un error en la siguiente operación de lectura o escritura (el kernel reporta `ECONNRESET` o devuelve 0 bytes). Esto permite al cliente saber que debe reconectarse. Bajo UDP, la caída del broker es completamente invisible para los clientes: el publisher seguirá enviando datagramas al vacío sin recibir ningún error, y el subscriber seguirá bloqueado en `recvfrom()` sin saber que el broker ya no existe. La recuperación en UDP requiere implementar un mecanismo de heartbeat a nivel de aplicación.

7.8. Sincronización de actualizaciones críticas

Garantizar que todos los subscribers reciban el mismo mensaje exactamente al mismo tiempo es un problema de difusión sincronizada. TCP dificulta esto porque cada conexión es independiente y el broker envía el mensaje a cada subscriber secuencialmente; los primeros subscribers reciben el mensaje antes que los últimos. UDP facilita esta sincronización mediante multicast o broadcast: un único datagrama puede ser entregado a múltiples destinos simultáneamente por la red, reduciendo la diferencia de tiempo entre recepciones. En el sistema implementado, el broker UDP envía secuencialmente a cada subscriber registrado, por lo que la sincronización perfecta tampoco está garantizada, aunque la diferencia temporal es mínima en una red local.

7.9. Uso de CPU y memoria en el broker

En el broker TCP, cada conexión activa mantiene estructuras de estado en el kernel (socket, buffers de envío/recepción, tabla de conexiones TCP), y en implementaciones con hilos como la versión en C, cada cliente requeriría un hilo adicional con su propio stack. En la implementación en assembly, el broker TCP procesa clientes de forma secuencial (sin hilos), lo que simplifica el modelo pero limita la concurrencia real. El broker UDP no mantiene estado por cliente: procesa un datagrama, lo reenvía y queda listo para el siguiente, con un footprint de memoria constante independiente del número de publishers activos.

7.10. Diseño de un sistema real para millones de usuarios

Para millones de usuarios simultáneos, la solución óptima sería una combinación de ambos protocolos según la criticidad de los datos. Los eventos críticos e irrecuperables (goles, tarjetas, resultado final) se transmitirían por TCP para garantizar entrega. Las actualizaciones de alta frecuencia y baja criticidad (estadísticas en tiempo real, posición del balón) se transmitirían por UDP para minimizar latencia y overhead. Esta arquitectura híbrida, similar a la utilizada por plataformas como ESPN o Sofascore, permite equilibrar confiabilidad y rendimiento según las necesidades específicas de cada tipo de dato.

8. Conclusiones

La implementación del sistema publicación–suscripción en x86-64 Assembly permitió observar de primera mano las diferencias fundamentales entre TCP y UDP a nivel de syscall del kernel de Linux. TCP demostró ser un protocolo robusto y confiable: todos los mensajes llegaron completos y en orden a ambos subscribers sin ninguna pérdida, gracias a su mecanismo de handshake, números de secuencia y confirmaciones ACK. UDP, por su parte, evidenció su naturaleza no confiable con la pérdida del 20 % de los mensajes, aunque su menor overhead de cabecera (8 bytes frente a 20 de TCP) y la ausencia de establecimiento de conexión lo hacen más eficiente para escenarios donde la velocidad prima sobre la exactitud.

El modelo publicación–suscripción demostró ser una arquitectura elegante y escalable para la distribución de información en tiempo real. El desacoplamiento entre publishers y subscribers, gestionado por el broker, permite agregar o eliminar componentes del sistema sin afectar al resto, lo que facilita el mantenimiento y la escalabilidad del sistema.

La implementación en lenguaje ensamblador x86-64 expuso la interacción directa con el kernel mediante instrucciones `syscall`, evidenciando que funciones de alto nivel como `socket()`, `bind()`, `connect()` y `sendto()` son en última instancia llamadas numéricas al sistema operativo. Esto reforzó la comprensión de la capa de abstracción que ofrecen lenguajes de alto nivel como C, y permitió apreciar el trabajo que realizan estas abstracciones de forma transparente para el programador.

Referencias

1. Kurose, J., & Ross, K. *Computer Networking: A Top-Down Approach*. Sección 2.7: Sockets Programming.

Archivos de Captura (.pcap)

Los archivos de captura de tráfico `tcp_pubsub.pcap` y `udp_pubsub.pcap` están disponibles en el siguiente enlace:

[Google Drive / GitHub – Archivos .pcap]